



GitHub e GitHub Desktop

Controle de Versão na Prática



O que é Controle de Versão?



Analogia: Google Docs com Histórico

Pense no Google Docs: você pode ver todas as alterações feitas, quem fez e quando. Pode voltar para qualquer versão anterior. O controle de versão faz isso com código-fonte, mas de forma muito mais poderosa e organizada.



Histórico completo

Cada alteração é registrada com autor, data e descrição. Você pode voltar a qualquer ponto no tempo.



Trabalho paralelo

Várias pessoas podem trabalhar ao mesmo tempo sem conflitos, usando branches (ramificações) independentes.



Segurança

Se algo quebrar, você pode reverter para uma versão estável. Nenhum código é perdido permanentemente.

</> Git vs GitHub: Qual a Diferença?

Git

Ferramenta (software)

- Sistema de controle de versão distribuído
- Funciona 100% local, no seu computador
- Criado por Linus Torvalds em 2005
- Linha de comando (terminal)
- Gratuito e open source
- Rastreia alterações em arquivos
- Permite branches e merges

GitHub

Plataforma (serviço web)

- Hospeda repositórios Git na nuvem
- Interface visual para gerenciar projetos
- Criado em 2008 (adquirido pela Microsoft)
- Colaboração: Pull Requests, Issues, Reviews
- Gratuito para repositórios públicos e privados
- GitHub Actions (CI/CD automático)
- GitHub Pages (hospedagem de sites)



GitHub Desktop: Git sem Terminal



Interface Gráfica

Aplicativo para Windows e macOS que permite usar Git de forma visual, sem precisar digitar comandos no terminal. Ideal para quem está começando.



Sincronização Fácil

Conecta diretamente com sua conta GitHub. Push, pull e fetch com um clique. Mantém seu repositório local e remoto sempre sincronizados.



Gestão de Branches

Crie, troque e faça merge de branches visualmente. Veja o histórico de commits em uma timeline interativa e resolva conflitos com editor integrado.

Download gratuito em: desktop.github.com | Windows e macOS | Open source

</> Terminal vs GitHub Desktop

Ação	Terminal (Git)	GitHub Desktop
Clonar repositório	<code>git clone <url></code>	File → Clone Repository
Ver alterações	<code>git status / git diff</code>	Painel Changes (automático)
Fazer commit	<code>git add . && git commit -m "msg"</code>	Selecionar + escrever + Commit
Enviar para o GitHub	<code>git push origin main</code>	Botão Push origin
Criar branch	<code>git checkout -b nova-branch</code>	Branch → New Branch
Resolver conflitos	Editar arquivos manualmente	Editor visual integrado



Quando usar cada um?

GitHub Desktop:

Ideal para iniciantes, tarefas do dia a dia, e quando você quer visualizar mudanças facilmente.

Terminal:

Necessário para operações avançadas (rebase, cherry-pick, stash, hooks) e automações em CI/CD.

Conceitos Fundamentais



Repositório (Repo)

Pasta do projeto que contém todos os arquivos e o histórico completo de alterações. Pode ser local (no seu PC) ou remoto (no GitHub).



Merge

Combinar as alterações de uma branch em outra. Exemplo: juntar a branch feature-login de volta na main após terminar o desenvolvimento.



Commit

Um "snapshot" do projeto em um momento específico. Cada commit tem uma mensagem descritiva, autor e timestamp. É a unidade básica do histórico.



Clone

Fazer uma cópia completa de um repositório remoto para o seu computador, incluindo todo o histórico de commits e branches.



Branch

Uma linha independente de desenvolvimento. A branch principal é a main (ou master). Branches permitem trabalhar em features sem afetar o código principal.



Push / Pull

Push envia seus commits locais para o GitHub. Pull baixa as alterações do GitHub para o seu computador. Mantém local e remoto sincronizados.



Fluxo de Trabalho com GitHub Desktop

1

Clone ou Crie

Clone um repositório existente do GitHub ou crie um novo repositório local.

2

Faça Alterações

Edite os arquivos do projeto no seu editor favorito (VS Code, etc).

3

Revise as Mudanças

O GitHub Desktop mostra automaticamente todos os arquivos alterados com diff visual.

4

Faça o Commit

Escreva uma mensagem descritiva e confirme as alterações no histórico local.

5

Push para o GitHub

Envie seus commits para o repositório remoto com um clique no botão Push.



Branches: Trabalhando em Paralelo

O que é uma Branch?

Uma branch é uma cópia independente do código onde você pode trabalhar sem afetar a versão principal. Quando terminar, você faz o merge (junção) de volta na branch principal. Isso permite que múltiplas features sejam desenvolvidas simultaneamente.

Convenções de Nome

<code>feature/login</code>	<i>Nova funcionalidade</i>
<code>fix/bug-cadastro</code>	<i>Correção de bug</i>
<code>hotfix/seguranca</code>	<i>Correção urgente</i>
<code>refactor/api-users</code>	<i>Refatoração de código</i>
<code>docs/readme-update</code>	<i>Atualização de docs</i>

No GitHub Desktop

Criar branch:

Branch → New Branch → digitar nome

Trocar de branch:

Clicar no seletor Current Branch no topo

Fazer merge:

Branch → Merge into Current Branch

Publicar branch:

Botão Publish Branch (envia para o GitHub)



Commits: Boas Práticas de Mensagens



Boas Mensagens

`feat: adiciona formulário de cadastro`

`fix: corrige validação de e-mail`

`docs: atualiza README com instruções`

`refactor: simplifica lógica de autenticação`

`style: ajusta espaçamento no header`

`test: adiciona testes para API de usuários`



Mensagens Ruins

`update`

`fix`

`alterações`

`asdkjf`

`commit final (de verdade agora)`

`WIP`

Dica: Use prefixos (feat, fix, docs, refactor, style, test) para categorizar commits



Pull Requests (PRs)

O que é um Pull Request?

Um Pull Request é uma solicitação para que as alterações de uma branch sejam revisadas e incorporadas em outra (geralmente a main). É o mecanismo central de colaboração no GitHub, permitindo code review, discussão e aprovação antes do merge.

1

Crie uma Branch

Trabalhe na sua feature ou correção em uma branch separada

2

Faça Commits

Salve suas alterações com mensagens descritivas

3

Abra o PR

No GitHub (ou via GitHub Desktop: Branch → Create Pull Request)

4

Revisão (Code Review)

Colegas revisam o código, sugerem mudanças e aprovam

5

Merge

Após aprovação, as alterações são incorporadas na branch principal



.gitignore: O que Não Versionar

O arquivo .gitignore lista padrões de arquivos e pastas que o Git deve ignorar. Esses arquivos não serão rastreados e não aparecerão nos commits. Essencial para manter o repositório limpo e seguro.

```
.gitignore (exemplo)

# Dependências
node_modules/
vendor/

# Variáveis de ambiente
.env
.env.local

# Build e compilados
dist/
build/
*.class

# Sistema operacional
.DS_Store
Thumbs.db
```

O que ignorar?

Sempre ignorar:

- Dependências (node_modules, vendor)
- Arquivos de ambiente (.env com senhas)
- Arquivos compilados (dist, build)
- Arquivos do SO (.DS_Store, Thumbs.db)
- Logs e caches

Nunca ignorar:

- Código-fonte do projeto
- Arquivo .gitignore em si
- Documentação (README, docs)
- Arquivos de configuração (package.json)



README.md: O Cartão de Visita do Projeto

README.md (exemplo)

Meu Projeto

Descrição curta do projeto.

Tecnologias

- Node.js
- Express
- MongoDB

Como Executar

```
```bash
npm install
npm start
```
```

Autores

- @seu-usuario

O que incluir?

- Nome e descrição do projeto
- Tecnologias utilizadas
- Pré-requisitos para rodar
- Instruções de instalação
- Como executar o projeto
- Exemplos de uso (screenshots)
- Estrutura de pastas
- Como contribuir
- Licença do projeto
- Autores e contato



Boas Práticas com GitHub



Commits pequenos e frequentes

Cada commit deve representar uma alteração lógica. Evite commits gigantes com várias mudanças.



Mantenha o README atualizado

O README é a primeira coisa que as pessoas veem. Mantenha instruções claras e atualizadas.



Use branches para features

Nunca trabalhe diretamente na main. Crie uma branch para cada feature ou correção.



Nunca commite senhas ou chaves

Use .env e .gitignore para proteger informações sensíveis. Se vazar, troque imediatamente.



Revise antes do merge

Use Pull Requests para revisão de código. Pelo menos uma pessoa deve revisar antes do merge.



Sincronize frequentemente

Faça pull regularmente para evitar conflitos grandes. Quanto mais tempo sem sincronizar, pior.